

Resumen para la presentaci3n del proyecto

AprenTIC Campus API 3 Gu3a de exposici3n y defensa t3cnica

1. Idea general del proyecto

AprenTIC Campus API es una API REST para gestionar la actividad acad3mica de un bootcamp multi-campus. Permite administrar alumnos, profesores, promociones, proyectos y notas, adem3s de obtener m3tricas acad3micas 3tiles para seguimiento.

- 3 Tecnolog3as principales: Node.js, Express, MongoDB Atlas y Mongoose.
- 3 Seguridad: autenticaci3n con JWT, roles de usuario y contrase3as cifradas con bcrypt.
- 3 Calidad y documentaci3n: Swagger, seed desde CSV y tests con Jest, Supertest y MongoDB en memoria.
- 3 Incluye un cliente web simple para demostrar login, CRUD y panel de analytics.

2. Organizaci3n del proyecto

El proyecto est3 separado por capas, lo que ayuda a mantener el c3digo ordenado y defendible:

```
src/  
app.js  
server.js  
config/  
models/  
routes/  
controllers/  
services/  
middlewares/  
docs/  
tests/  
  
scripts/  
data/  
client/
```

- 3 routes: definen las URLs de la API y aplican middlewares.
- 3 controllers: reciben la petici3n HTTP, llaman al service y devuelven respuesta JSON.
- 3 services: contienen la l3gica de negocio y las operaciones con MongoDB.
- 3 models: definen los esquemas de datos con Mongoose.
- 3 middlewares: controlan autenticaci3n, permisos y errores.
- 3 docs: configuraci3n de Swagger.
- 3 tests: pruebas unitarias e integraci3n.
- 3 scripts y data: carga inicial de datos desde CSV.
- 3 client: interfaz visual para consumir la API.

3. Archivos principales

src/server.js

Es el punto de entrada. Conecta con MongoDB mediante connectDB() y arranca el servidor en el puerto configurado.

src/app.js

Configura Express: CORS, JSON, Swagger, rutas principales y middleware global de errores. También importa los modelos para registrar sus relaciones.

src/config/env.js

Carga variables de entorno: PORT, MONGODB_URI y JWT_SECRET.

src/config/db.js

Gestiona la conexión con MongoDB usando Mongoose. Si la conexión falla, muestra el error y detiene el proceso.

4. Modelos de datos

Usuario.js

Representa usuarios que pueden iniciar sesión. Contiene email, password cifrada y rol: admin o profesor.

Profesor.js

Guarda nombre, email y una referencia opcional al usuario asociado.

Alumno.js

Guarda nombre, email, foto opcional y una referencia a la promoción a la que pertenece.

Promoción.js

Representa una promoción del bootcamp, con nombre, fechas de inicio y fin, y referencia al campus.

Campus.js

Normaliza los campus. Aunque no tiene CRUD propio, es clave para métricas como aptos por campus y evita duplicar nombres.

Proyecto.js

Representa proyectos asociados a una promoción. Incluye un array de notas, donde cada nota tiene alumno, nota, estado y profesor.

5. Rutas de la API

? /api/auth: registro y login.

? /api/alumnos: CRUD de alumnos.

? /api/profesores: CRUD de profesores.

? /api/promociones: CRUD de promociones.

? /api/proyectos: CRUD de proyectos y gestión de notas.

? /api/analytics: métricas académicas.

La mayoría de rutas están protegidas con JWT. Las operaciones sensibles usan además control de roles.

6. Controllers

Los controllers hacen de puente entre la capa HTTP y la lógica de negocio. Su función es leer req.params o req.body, llamar al service correspondiente, devolver JSON y enviar errores al middleware global mediante next(err).

? Ejemplo: alumnos.controller.js llama a alumnosService.obtenerAlumnos(), crearAlumno(), actualizarAlumno() o eliminarAlumno().

? También gestionan respuestas 404 cuando un recurso no existe.

7. Services y l?gica de negocio

auth.service.js

Registra usuarios, comprueba duplicados, cifra contrase?as con bcrypt y genera tokens JWT en el login.

alumnos.service.js

CRUD de alumnos. Usa populate("promocion") para devolver la promoci?n completa.

profesores.service.js

CRUD de profesores. Al crear un profesor, tambi?n crea un usuario asociado con rol profesor y contrase?a cifrada.

promociones.service.js

CRUD de promociones. Usa populate("campus") para devolver la informaci?n completa del campus.

proyectos.service.js

Gestiona proyectos y notas. Al crear un proyecto para una promoci?n, busca los alumnos de esa promoci?n y crea autom?ticamente notas en estado Pendiente. Tambi?n permite a?adir o actualizar notas.

analytics.service.js

Es una de las partes m?s potentes del proyecto. Usa agregaciones de MongoDB para calcular m?tricas cruzando proyectos, alumnos, promociones y campus.

? aptosPorCampus: calcula el porcentaje de aptos agrupado por campus.

? alumnosEnRiesgo: lista alumnos con notas inferiores a 5.

? rankingNoAptos: ordena proyectos por n?mero de No Apto.

8. Middlewares

auth.required.js

Comprueba que la petici?n incluya un token JWT v?lido en Authorization. Si es correcto, a?ade los datos del usuario a req.user.

role.required.js

Comprueba si el rol del usuario est? autorizado para acceder a una ruta. Si no lo est?, devuelve 403.

errorHandler.js

Centraliza los errores de la API y devuelve una respuesta JSON uniforme.

9. Swagger y documentaci?n

Swagger est? configurado en src/docs/swagger.js y disponible en /api-docs. Define esquemas de datos, endpoints y seguridad Bearer JWT. En la demo se puede usar para ense?ar y probar la API.

10. Seed de datos

scripts/seed.js carga datos iniciales desde data/alumnos.csv. Limpia la base de datos, crea campus, promociones, profesores, usuarios, alumnos y proyectos con notas.

? Usuario demo admin: admin@aprentic.com

? Contrase?a admin: admin123

? Contrase?a de profesores creados por seed: profesor123

11. Cliente web

El cliente en `client/` permite mostrar el proyecto visualmente. Tiene login, dashboard, navegaci3n por secciones, modales para CRUD, gesti3n de notas y carga de analytics.

- ? `client/index.html`: estructura de pantallas y secciones.
- ? `client/js/main.js`: llamadas fetch a la API, gesti3n de token y l3gica de interfaz.
- ? `client/styles/style.css`: dise2o visual del dashboard.

12. Tests

El proyecto incluye pruebas unitarias e integraci3n. Se han ejecutado correctamente: 3 suites y 7 tests pasados.

- ? `alumnos_service_test.js`: prueba el service de alumnos con mocks.
- ? `require_role_test.js`: prueba el control de permisos por rol.
- ? `auth_test.js`: prueba registro y login con Supertest y MongoDB en memoria.

13. Flujo completo para explicar

- ? El usuario inicia sesi3n con email y contrase2a.
- ? El backend valida credenciales, compara la contrase2a cifrada y devuelve un JWT.
- ? El frontend guarda el token en `localStorage`.
- ? Cada petici3n protegida env2a `Authorization: Bearer token`.
- ? `authRequired` valida el token y `requireRole` valida permisos cuando hace falta.
- ? La ruta llama al controller, el controller llama al service y el service usa modelos Mongoose.
- ? MongoDB devuelve los datos y la API responde en JSON.
- ? El cliente actualiza el dashboard.

14. Puntos fuertes para defender

- ? Arquitectura clara por capas.
- ? Autenticaci3n JWT y control de roles.
- ? Contrase2as cifradas con `bcrypt`.
- ? Modelado con referencias entre colecciones.
- ? Uso de `populate` para enriquecer respuestas.
- ? Agregaciones reales de MongoDB para analytics.
- ? Carga inicial desde CSV.
- ? Documentaci3n Swagger.
- ? Tests unitarios e integraci3n.
- ? Cliente web funcional para demo.

15. Preguntas probables y respuestas

?Qu3 diferencia hay entre controller y service?

El controller gestiona HTTP: petici3n, respuesta y c3digos de estado. El service contiene la l3gica de negocio y acceso a base de datos.

¿Por qué usar JWT?

Porque permite proteger rutas sin guardar sesiones en servidor. El token viaja en cada petición y contiene datos básicos del usuario.

¿Por qué bcrypt?

Porque no se deben guardar contraseñas en texto plano. bcrypt guarda un hash seguro y permite comparar la contraseña introducida.

¿Por qué existe Campus como modelo?

Porque el proyecto necesita métricas por campus. Tenerlo normalizado evita duplicar datos y facilita consultas.

¿Qué hace populate?

Convierte referencias ObjectId en objetos completos. Por ejemplo, un alumno guarda el ID de promoción, pero la API puede devolver la promoción completa.

¿Qué parte destacarías como más compleja?

Las agregaciones de analytics, porque cruzan varias colecciones usando unwind, lookup, group y project.

¿Qué mejorarías en una versión futura?

Añadir validaciones con express-validator, crear CRUD específico de campus, ampliar tests de proyectos y analytics, añadir paginación y mejorar la sincronización entre profesor y usuario.

16. Guion corto de exposición

¿Mi proyecto es una API REST para gestionar un bootcamp multi-campus. Está organizado por capas: rutas, controllers, services, models y middlewares. La autenticación se hace con JWT y las contraseñas se guardan cifradas con bcrypt. Los datos se modelan con Mongoose y están relacionados mediante referencias. Además de los CRUD básicos, he añadido analytics con agregaciones de MongoDB para obtener indicadores académicos. También incluí Swagger para documentar la API, un seed desde CSV y tests automáticos para comprobar autenticación, roles y servicios.